Table of Contents

Library Design and Architecture Choices

In this project we implemented a small neural-network library from scratch using only NumPy.
Our main goal was to separate responsibilities into clear modules so that the same code base can be reused for simple problems such as XOR and also for more complex models such as the MNIST autoencoder and the latent-space SVM.

Project structure

The library is organized in a Python package called **lib/:**

- **lib/layers.py** – base Layer class and the fully connected Dense layer.
- **lib/activations.py** – non-linear activation layers (ReLU, Sigmoid, Tanh).
- **lib/losses.py** – loss functions and their gradients (mean squared error in our case).
- **lib/network.py** – the Sequential model that stores the layers and runs training.
- **lib/optimizer.py** – placeholder for optimizers (we use simple SGD style updates).

Outside the library we have:

- **notebooks**/ – Jupyter notebooks for gradient checking, XOR, autoencoder and SVM experiments.
- **report**/ – the written report (project_report.pdf).
- **requirements.txt** – lists the Python dependencies for reproducibility.

This separation makes the **library code** independent from the **experiments and report**, similar to how real ML projects are structured.

Layer and activation design

All layers inherit from a simple abstract Layer class that defines two methods:

- forward(input) – computes the output of the layer.
- backward(grad_output, learning_rate) – propagates gradients and updates parameters.

The Dense layer implements a fully connected transformation

$$\boldsymbol{z} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}$$

and stores the input so that it can compute gradients during backpropagation.
Activation functions (Tanh, Sigmoid, ReLU) are implemented as separate layers.
This design lets us build different architectures just by changing the sequence of layers, for example:

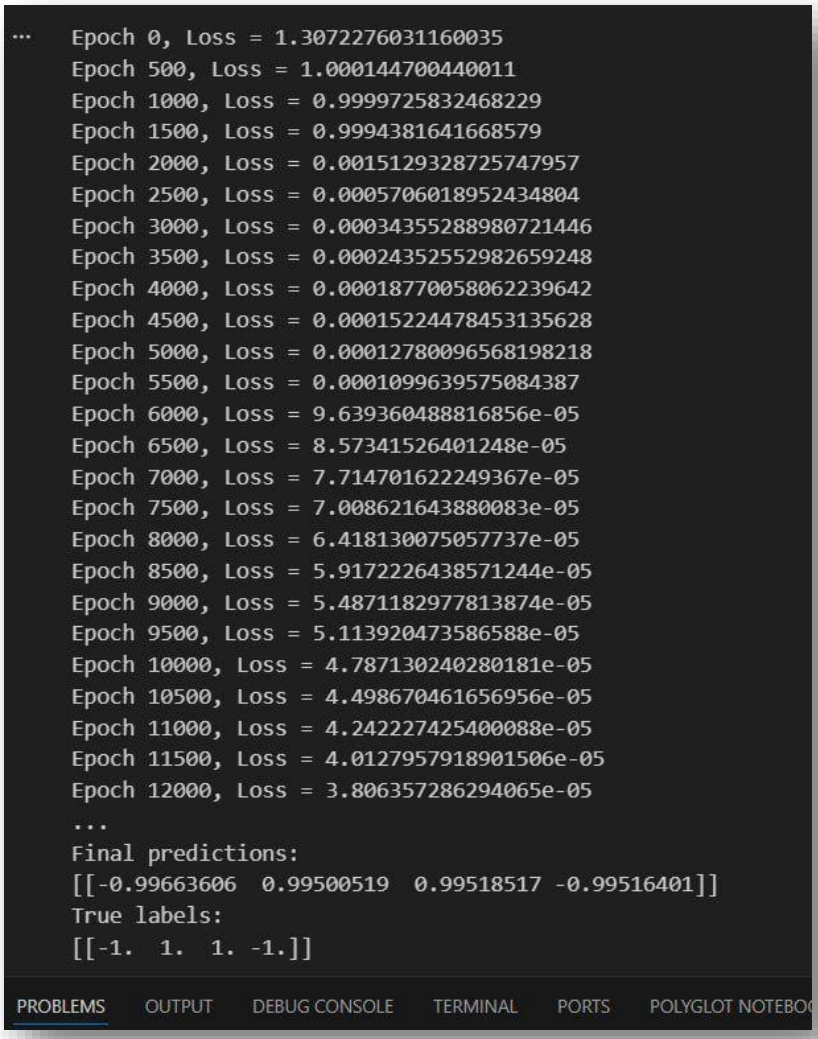
- **XOR network**: Dense → Tanh → Dense → Tanh
- **Autoencoder**: Dense → Tanh → … → Dense → Tanh

Keeping activations as layers also makes the backward pass cleaner, because each activation knows its own derivative.

Loss and gradient checking

The mean squared error loss (mse_loss) and its analytical gradient (mse_grad) are implemented in losses.py.
During training the Sequential model calls the loss function and then uses its gradient as the starting point for backpropagation.
To verify that our backpropagation implementation is correct, we added **numerical gradient checking** in the notebook.
The Dense layer stores its parameter gradients (dW, db) during the backward pass so we can compare them with numerical gradients computed with

XOR test output



Github Link:

https://github.com/MonMon204/CI_Project.git